

Programmation Génétique (Genetic Programming, GP)

Introduction

La **Programmation Génétique** (GP) a été introduite et développée par John Koza au début des années 1990.

L'objectif principal est d'appliquer les concepts évolutionnistes à des programmes informatiques, plutôt qu'à un ensemble de valeurs numériques.

On cherche à faire évoluer un programme de manière à ce qu'il puisse résoudre un problème donné de manière optimale, ouvrant ainsi la voie à la robotique évolutionniste.

Pour cela, on considère une population de programmes et on applique des opérateurs de sélection et de recombinaison (dont le crossover) afin d'obtenir le programme désiré. Au lieu d'optimiser des paramètres numériques, la GP fait évoluer les programmes eux-mêmes jusqu'à ce qu'ils accomplissent la tâche souhaitée.

Un tel programme doit :

- Produire une sortie correcte pour toute entrée possible
- Découvrir la fonction liant les données d'entrée aux données de sortie

Il peut être appliqué à :

- L'ajustement de courbes algébriques, la régression symbolique, la classification
- La construction de modèles de trading en finance
- La détermination de conditions aux limites à partir de données empiriques
- Le contrôle du mouvement de robots dans des environnements complexes

Ce processus est typique de l'apprentissage automatique (ML), mais peut aussi être vu comme une tâche d'optimisation : minimiser la différence entre la sortie spécifiée et celle obtenue par la GP. Cela montre la frontière ténue entre ML et optimisation.

Régression Symbolique

Le but de la **régression symbolique** (SR) est de construire un programme qui relie les données d'entrée aux données de sortie sans prédéfinir la forme fonctionnelle.

La SR explore l'immense espace de toutes les expressions symboliques possibles, construites à partir d'un ensemble de fonctions et de variables.

On utilise un ensemble d'apprentissage (training set) composé de paires (entrée, sortie) pour générer des programmes. Le but est qu'un programme construit sur cet ensemble soit capable de généraliser, c'est-à-dire de détecter la relation sous-jacente entre les données d'entrée et de sortie.

Cette généralisation est testée sur un ensemble de test (test set) qui ne fait pas partie de l'ensemble d'apprentissage. Il existe un risque de surapprentissage (overfitting) : excellente performance sur l'ensemble d'apprentissage, mais très mauvaise sur l'ensemble de test.

La SR est puissante car :

- Elle n'a pas de structure de modèle prédéfinie
 - Elle peut découvrir : polynômes, expressions rationnelles, exponentielles, compositions imbriquées, modèles hybrides
 - Elle est interprétable : contrairement aux réseaux de neurones, elle produit des formules explicites et lisibles
 - Elle permet la construction automatique de caractéristiques (features)
 - Elle explore naturellement des mappings non linéaires
-

Inférence de Comportement

Lorsque le but est de déterminer le comportement d'un robot (ou d'un groupe de robots) à partir de sa perception de l'environnement, on peut simuler le comportement induit par la GP et évaluer sa performance dans des environnements générés aléatoirement.

On mesure le taux de succès ou d'échec pour les tâches attendues. Cette approche est liée à l'apprentissage par renforcement (Reinforcement Learning, RL).

Codage des Programmes

Défis et Approches

Le défi principal est de coder un programme de manière à pouvoir lui appliquer des opérateurs évolutionnistes tout en générant des individus valides (programmes syntaxiquement corrects et exécutables).

Deux approches distinctes sont généralement utilisées :

- Les arbres (tree-based)
- Les programmes linéaires basés sur une pile (stack-based linear programs)

Dans tous les cas, l'espace de recherche (l'espace de tous les programmes possibles) est limité par la nature des langages de programmation existants.

Calcul de la Fitness (Régression Symbolique)

Soit p un programme tel que $y = p(x)$ est la sortie générée par p pour l'entrée x . Notez que x et y peuvent être des structures de données.

Soit $A = \{\langle x_1, y_1 \rangle, \dots, \langle x_k, y_k \rangle\}$ un ensemble d'apprentissage composé de k paires (entrée, sortie).

La fitness (ou fonction de perte) du programme p est définie par :

$$f(p) = \sum_{i=1}^k |y_i - p(x_i)|$$

C'est la capacité du programme p à générer la sortie attendue à partir de l'entrée donnée.

Dans la majorité des cas en GP, on cherche une représentation algébrique d'une expression booléenne ou arithmétique. Par exemple, à partir d'observations $\{(x_i, y_i)\}$, on cherche une relation $y = g(x)$ où g est donnée algébriquement, par exemple $g(x) = 2x^2 - 3x + 1$.

Important : Essayer de faire évoluer des programmes écrits en C, Java, Python, etc. à l'aide d'opérations de crossover et de mutation est voué à l'échec car la plupart des recombinaisons produiraient des programmes syntaxiquement invalides.

Codage par S-expressions (Arbres)

Koza a proposé de coder les programmes avec un langage fonctionnel (similaire à Lisp), basé sur des instructions appelées S-expressions (expressions symboliques).

Par exemple, $(+x2)$ signifie $x + 2$. L'expression $2x^2 + 3x + 1$ devient :

$$(+(*(*xx)2)(+(*3x)1))$$

Si on remplace un opérateur dans une S-expression par une autre S-expression, on obtient une commande valide. On peut donc définir des opérateurs génétiques (crossover et mutation) sur les S-expressions.

Le codage par arbre est équivalent aux S-expressions, mais plus facile à comprendre.

Ensembles de Fonctions et de Terminaux

L'espace de recherche des GP est spécifié par un ensemble F de fonctions et un ensemble T de terminaux.

- F contient toutes les opérations possibles (par exemple, $F = \{+, -, *, /\}$)
- T contient toutes les variables et constantes possibles (par exemple, $T = \{A, B, C, D\}$)

Pour les fonctions booléennes, on pourrait avoir $F = \{\text{AND}, \text{OR}, \text{NOT}\}$ et $T = \{b_1, b_2, b_3, \dots, \text{TRUE}, \text{FALSE}\}$.

On peut aussi inclure des opérateurs de contrôle, comme IF. Par exemple, (IF A B C D) est une fonction à 4 arguments : si $A == B$, alors retourne C, sinon D.

Propriété de Clôture (Closure Property)

Chaque fonction doit être capable d'accepter comme argument toutes les valeurs retournées par d'autres fonctions et toutes les valeurs de l'ensemble terminal T .

Des opérateurs comme la division doivent être augmentés pour accepter toute paire d'arguments (par exemple, gérer la division par zéro).

L'espace de tous les programmes admissibles est constitué de toutes les compositions de fonctions disponibles à partir de F avec des éléments terminaux de T .

Cet espace a potentiellement une dimension infinie et la taille des individus est arbitraire. Il est pratiquement nécessaire de limiter leur taille. Certains morceaux de programmes peuvent ne rien faire (par exemple, $(-xx)$ qui peut être simplifié en 0), c'est le problème de "bloat".

La propriété de clôture est une contrainte fondamentale pour la GP. Des exemples de problèmes incluent la division par zéro, l'incompatibilité de type, les erreurs de domaine, et les structures de contrôle mal utilisées (par exemple, $(if\ x\ y\ z)$ avec x numérique et non booléen).

Population Initiale

À partir des ensembles F et T , on peut générer une population initiale de programmes valides.

Chaque nœud est tiré aléatoirement de F ou T . Si c'est un terminal, la branche se termine. Sinon, il faut tirer autant d'éléments que requis par l'opérateur de F .

On peut biaiser le choix entre F et T en fonction de la profondeur : près de la racine, on a une probabilité plus élevée de choisir une fonction, puis une probabilité plus élevée de tirer un terminal en s'éloignant de la racine. Ces probabilités définissent la profondeur moyenne des programmes.

Opérateurs Génétiques (Arbres)

Crossover (à la Koza) : On choisit aléatoirement un lien dans l'arbre représentant chaque parent et on échange les sous-arbres correspondants. Les points de crossover sont typiquement choisis avec une probabilité non uniforme pour favoriser les nœuds internes par rapport aux feuilles.

Mutation (à la Koza) : On sélectionne aléatoirement un sous-arbre du programme et on le remplace par un autre sous-arbre généré aléatoirement à partir de F et T .

Exemple : Recherche de Modèles de Trading Optimaux

But : Faire évoluer un programme qui formule des recommandations d'investissement sur le marché des changes (forex).

On dispose d'indicateurs de marché $\{I_k(t)\}$, $k = 1, \dots, m$, qui sont typiquement des moyennes mobiles des taux de change à différentes échelles de temps.

On cherche un programme qui prend les $\{I_k(t)\}$ en entrée et produit trois sorties possibles : 0 (ne rien faire), -1 (vendre), 1 (acheter).

La fitness est définie comme le rendement moyen généré par ce modèle s'il avait été utilisé sur les années passées, auquel on soustrait le risque associé mesuré par sa variance.

Un exemple d'arbre obtenu pourrait utiliser des indicateurs complexes I_1, I_2, I_3, I_4 basés sur des moyennes mobiles de séries temporelles longues, avec des constantes empiriques. Le rendement est bon, mais pas exceptionnel.

Programmes Génétiques Linéaires Basés sur une Pile

Principe

Pour suivre l'approche des langages procéduraux, on code les GP avec une série d'instructions exécutées séquentiellement. Pour garantir la validité syntaxique après crossover/mutation, on utilise une approche basée sur une pile.

Par exemple :

- Instruction A : place la valeur de la variable A dans la pile
- Instruction B : place la valeur de B dans la pile
- Instruction ADD : retire les deux éléments du sommet de la pile, les additionne et empile le résultat

Ainsi, le programme A B ADD C MUL 2 sqrt DIV correspond à :

$$\frac{(A + B) \times C}{\sqrt{2}}$$

Les opérations doivent être adaptées pour tolérer le manque d'arguments dans la pile.

Opérateurs Génétiques (Linéaire)

Crossover : Similaire au crossover sur une chaîne de bits. On choisit un point de crossover dans deux programmes et on échange les parties.

Exemple :

- P1 : A B ADD | C MUL 2 sqrt DIV
- P2 : 1 B SUB | A A MUL ADD 3

Les descendants sont :

- P1' : A B ADD A A MUL ADD 3
- P2' : 1 B SUB C MUL 2 sqrt DIV

P1' laisse dans la pile deux valeurs : $A + B + A^2$ et 3. P2' laisse une valeur : $(1 - B) * C / \sqrt{2}$.

Mutation : On change aléatoirement une instruction en une autre.

Exemple : A B ADD C MUL 2 sqrt DIV devient A B MUL C MUL 2 sqrt DIV, qui laisse dans la pile $(A * B) * C / \sqrt{2}$.

Exemples

Exemple booléen : On cherche l'expression algébrique de la fonction booléenne $f(x_0, x_1, x_2) = x_0 \text{ AND } x_1 \text{ AND } x_2$.

Avec $T = \{X_0, X_1, X_2\}$ et $F = \{\text{AND}, \text{OR}, \text{NOT}\}$, on considère des programmes de 5 instructions. Avec 6 choix par instruction (3 terminaux + 3 fonctions), on a $6^5 = 7776$ programmes possibles.

La fitness est le nombre de lignes de la table de vérité correctement calculées (optimum = 8).

Exemple arithmétique : On cherche à approximer la fonction $F(x_0, x_1) = (x_0 + x_1)^2$.

Avec $F = \{\text{ADD}, \text{MUL}, \text{DUP}, \text{SUB}\}$ et $T = \{X_0, X_1\}$, et un ensemble d'apprentissage de paires $(x_0, x_1, F(x_0, x_1))$, on minimise la somme des valeurs absolues des différences.

Contrôle de Flux

On peut définir des structures de contrôle comme des instructions conditionnelles IF ou des boucles.

IF : Prend l'élément du sommet de la pile α et le compare à 0. Si $\alpha \geq 0$, les instructions suivantes sont exécutées normalement jusqu'à ENDIF. Sinon, elles sont ignorées.

Boucles : On utilise une pile de contrôle en plus de la pile de données. La pile de contrôle contient la position de l'instruction LOOP et le nombre d'itérations (pris du sommet de la pile de données).

Quand on rencontre END-LOOP, on décrémente le nombre d'itérations et on revient à la position LOOP, sauf si le nombre d'itérations est nul. On travaille avec le sommet de la pile de contrôle, permettant des boucles imbriquées.

Exemple : Programme [X1, LOOP, X0, ADD, END-LOOP] pour calculer la somme de X0 répétée X1 fois (c'est-à-dire $X0 * X1$). Avec $X0=3$ et $X1=2$, on obtient 6.

Guided Parameters (Paramètres Guidés)

- **Fonction set (F) et Terminal set (T)** : Choix des opérations et variables/constantes autorisées.
- **Taille de la population** : Nombre de programmes dans la population.
- **Nombre de générations** : Combien d'itérations de l'évolution.
- **Probabilités des opérateurs** : Probabilités d'appliquer le crossover, la mutation, etc.
- **Méthode de sélection** : Par exemple, sélection par tournoi, roulette, etc.
- **Limites de taille/depth** : Pour contrôler le "bloat" (taille maximale des arbres, profondeur maximale).
- **Critère d'arrêt** : Fitness cible atteinte, nombre maximal de générations, stagnation, etc.
- **Fonction de fitness** : Définition de la mesure de performance (par exemple, erreur absolue moyenne, précision, etc.).
- **Taux de mutation** : Probabilité de muter un nœud.
- **Taux de crossover** : Probabilité d'effectuer un crossover.
- **Méthode de génération de la population initiale** : Aléatoire, avec biais de profondeur, etc.

Ces paramètres doivent être ajustés en fonction du problème pour obtenir de bonnes performances.